

Министерство образования Иркутской области
Государственное бюджетное профессиональное
образовательное учреждение Иркутской области
«Иркутский авиационный техникум»
(ГБПОУИО «ИАТ»)

КР.09.02.075.26.231.22 ПЗ

БЛОГ МУЗЫКИ

Председатель ВЦК:	_____	(М.А. Кудрявцева)
	(подпись, дата)	
Руководитель:	_____	(М.А. Кудрявцева)
	(подпись, дата)	
Студент:	_____	(Г.Д. Щемелев)
	(подпись, дата)	

Иркутск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1. Описание предметной области	5
2. Анализ инструментальных методов разработки	7
3. Техническое задание	10
4 Проектирование музыкального блога	11
4.1 Структурная схема музыкального блога.....	11
4.2 Функциональная схема музыкального блога.....	15
4.3 Проектирование базы данных	20
4.4 Проектирование интерфейса	23
5 Разработка музыкального блога	26
5.1 Разработка интерфейса музыкального блога.....	26
5.2 Разработка базы данных музыкального блога.....	27
5.3 Разработка музыкального блога	30
6 Документирование программного продукта	34
6.1 Руководство пользователя музыкального блога.....	34
ЗАКЛЮЧЕНИЕ.....	40
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	42
Приложение А – Техническое задание	43
Приложение Б – Листинг кода контроллера профиля	50

ВВЕДЕНИЕ

В современном мире информационные технологии играют важную роль в жизни общества и активно применяются в различных сферах деятельности человека. Одной из таких сфер является музыкальное творчество. Благодаря развитию интернета и веб-технологий у музыкантов появилась возможность самостоятельно публиковать свои произведения, делиться ими с широкой аудиторией и получать обратную связь от слушателей без участия профессиональных студий звукозаписи и музыкальных лейблов.

Особенно актуальными становятся интернет-платформы, ориентированные на начинающих и независимых исполнителей. Такие ресурсы позволяют авторам размещать собственные музыкальные треки, а слушателям – знакомиться с новым контентом, оценивать его и выражать своё мнение. Наличие системы оценок помогает формировать представление о популярности и качестве музыкальных произведений, а также способствует развитию авторов за счёт обратной связи.

Музыкальные блоги и специализированные вебсайты выполняют не только развлекательную, но и образовательную функцию. Они способствуют развитию музыкального вкуса, формированию творческих сообществ и обмену опытом между пользователями.

Разработка музыкального блога музыкального блога требует учета особенностей работы с аудиофайлами, пользовательскими данными и механизмами взаимодействия между участниками системы. Кроме того, система должна быть простой в использовании, понятной для пользователей и обеспечивать базовый уровень защиты данных.

Актуальность данного курсовой работы обусловлена необходимостью создания доступной и функциональной информационной системы, которая позволит пользователям публиковать музыкальные треки, прослушивать работы других авторов и оценивать их. Подобная система может применяться в учебных целях, для развития творческих навыков и в качестве демонстрационного примера веб-приложения.

Цель курсовой работы – разработка музыкального блога, предназначенного для публикации и оценки музыкальных треков.

Для достижения поставленной цели в ходе работы необходимо решить следующие задачи:

- Изучить особенности предметной области музыкальных онлайн платформ;
- Определить требования к функционалу музыкального блога.
- Выбрать программные средства и инструменты разработки.
- Разработать структуру базы данных.
- Спроектировать программный продукт.
- Реализовать программный продукт.
- Подготовить пользовательскую документацию.

1 Описание предметной области

Предметной областью курсовой работы является автоматизация блога музыки, предназначенного для публикации, хранения и оценки музыкальных треков пользователями.

Блог музыки представляет собой веб-ресурс, на котором пользователи могут размещать собственные музыкальные произведения, прослушивать работы других авторов и взаимодействовать с ними с помощью системы оценок. Основной задачей такой системы является создание удобной платформы для творческого обмена музыкальным контентом и получения обратной связи от аудитории.

В отличие от профессиональных стриминговых сервисов, музыкальный блог ориентирован на начинающих и независимых исполнителей. Основное внимание уделяется простоте использования, возможности свободной публикации контента и взаимодействию между пользователями без сложных требований и ограничений.

Музыкальный трек – это цифровой аудиофайл, загружаемый пользователем в систему. В рамках данного сервиса музыкальные треки могут быть представлены в популярных форматах (MP3, WAV и других). Каждый трек сопровождается основной информацией, необходимой для его идентификации и отображения в системе. К такой информации относятся название трека, имя автора, жанр, дата публикации и краткое описание.

Жанр музыкального трека используется для удобства навигации и поиска контента. Разделение треков по жанрам позволяет пользователям быстрее находить интересующую их музыку и упрощает структуру музыкального блога.

В рамках работы системы предусмотрено несколько категорий пользователей, каждая из которых обладает определёнными правами доступа.

Гость – пользователь, не прошедший процедуру регистрации и авторизации. Гость имеет возможность посещать страницы музыкального блога, просматривать список опубликованных треков, прослушивать музыкальные композиции и просматривать оценки других пользователей. При этом гость не имеет возможности загружать собственные треки и оставлять оценки.

Зарегистрированный пользователь – пользователь, создавший учетную запись в системе. После регистрации и входа в систему пользователь получает расширенные возможности. Он может загружать собственные музыкальные треки, редактировать информацию о них, удалять свои публикации, а также прослушивать и оценивать треки других пользователей. Оценка трека выражается в виде числового значения и используется для формирования общего рейтинга композиции.

Процесс оценки музыкальных треков является важной частью работы системы. Музыкальный блог позволяет каждому зарегистрированному пользователю поставить оценку опубликованному треку. На основе выставленных оценок формируется средний рейтинг, который отображается на странице трека и помогает другим пользователям ориентироваться при выборе музыкального контента.

Администратор – пользователь с расширенными правами доступа. Администратор осуществляет контроль за работой музыкального блога и отвечает за корректность размещаемого контента. В обязанности администратора входит управление учетными записями пользователей, удаление или блокировка недопустимого контента, а также контроль структуры данных в системе. Администратор имеет доступ к административной панели, через которую осуществляется управление системой.

2 Анализ инструментальных методов разработки

Для разработки блога необходимо определить перечень инструментов разработки. Выбор инструментальных средств оказывает прямое влияние на удобство разработки, стабильность работы системы и возможность её дальнейшего развития.

Для реализации серверной части блога были рассмотрены (таблица 1) следующие языки программирования: Python, PHP и C#.

Python – высокоуровневый язык программирования, отличающийся простым и понятным синтаксисом. Язык широко применяется в веб-разработке, автоматизации и создании серверных приложений.

PHP – язык программирования, ориентированный на создание серверных веб-приложений и динамических сайтов.

C# – объектно-ориентированный язык программирования, применяемый для разработки веб-приложений, десктопных программ и корпоративных систем на платформе .NET. В таблице 1 представлено сравнение выбранных языков программирования.

Таблица 1 – Сравнение языков программирования

Характеристика	Python	PHP	C#
Основное назначение	Веб-разработка, серверные приложения	Серверная веб-разработка	Корпоративные и веб-приложения
Типизация	Динамическая	Динамическая	Статическая
Сложность освоения	Низкая	Средняя	Средняя
Читаемость кода	Высокая	Средняя	Высокая
Популярность в веб-разработке	Высокая	Высокая	Средняя

По результатам анализа был выбран язык программирования Python, так как он прост в изучении, хорошо подходит для учебных проектов и широко используется совместно с веб Фреймворками.

Для разработки веб-приложения на языке Python были рассмотрены следующие фреймворки: Django и Flask (таблица 2).

Django – полнофункциональный веб Фреймворк, включающий в себя встроенную систему аутентификации, ORM, административную панель и средства работы с базой данных.

Flask – микро фреймворк, предоставляющий базовые возможности для веб-разработки и требующий дополнительной настройки для расширенного функционала.

Таблица 2 – Сравнение фреймворков Django и Flask

Характеристика	Django	Flask
Тип фреймворка	Полнофункциональный	Микрофреймворк
Встроенная работа с БД	Есть	Отсутствует
Административная панель	Есть	Отсутствует
Удобство для учебных проектов	Высокое	Среднее
Скорость разработки	Высокая	Средняя

По результатам анализа был выбран фреймворк Django, так как он предоставляет готовые инструменты для быстрой разработки и удобен для создания информационных систем.

Для написания и отладки программного кода были рассмотрены следующие среды разработки: PyCharm и Visual Studio Code (таблица 3).

PyCharm – интегрированная среда разработки для языка Python, предоставляющая инструменты для работы с проектами, отладки, тестирования и интеграции с фреймворками.

Visual Studio Code – универсальный редактор кода с поддержкой различных языков программирования и расширений.

Таблица 3 – Сравнение сред разработки

Характеристика	PyCharm	Visual Studio Code
Специализация	Python	Универсальная
Поддержка Django	Встроенная	Через расширения
Удобство отладки	Высокое	Среднее
Сложность освоения	Низкая	Низкая
Функциональность	Высокая	Средняя

В качестве среды разработки была выбрана PyCharm, так как она специализирована для работы с Python и Django и облегчает процесс разработки.

Для хранения данных музыкального блога была рассмотрена система управления базами данных MySQL и SQLite (таблица 4).

MySQL – клиент-серверная реляционная СУБД, широко используемая в веб-приложениях и обеспечивающая высокую надёжность.

SQLite – встраиваемая СУБД, подходящая для небольших локальных приложений.

Таблица 4 – Сравнение СУБД MySQL и SQLite

Характеристика	MySQL	SQLite
Тип СУБД	Клиент-серверная	Встраиваемая
Масштабируемость	Высокая	Низкая
Поддержка многопользовательской работы	Есть	Ограниченная
Использование в веб-приложениях	Широкое	Ограниченное
Надёжность хранения данных	Высокая	Средняя

На основании проведённого анализа была выбрана СУБД MySQL, так как она подходит для многопользовательских веб-приложений и обеспечивает надёжное хранение данных.

По результатам анализа для реализации блога музыки был выбран следующий стек инструментальных средств:

- Язык программирования – Python.
- Веб-фреймворк – Django.
- Среда разработки – PyCharm.
- Система управления базами данных – MySQL.
- Для моделирования архитектуры (структурные схемы, диаграммы потоков данных) – Draw.io.
- Для разработки прототипа музыкального блога и детального дизайна – Figma.

3 Техническое задание

В начале разработки создавалось техническое задание, в котором указывались основные требования.

Для создания технического задания использовался стандарт ГОСТ 34.6022020.

Согласно ГОСТ 34.6022020 техническое задание должно включать следующие разделы:

1. Введение.
2. Основания для разработки.
3. Назначение музыкального блога.
4. Требования к музыкальному блогу.
5. Требования к техническому обеспечению.
6. Требования к программному обеспечению.
7. Организационно-технические требования.

Техническое задание на разработку музыкального блога представлено в приложении А.

4 Проектирование музыкального блога

4.1 Функциональные схемы музыкального блога

Проектирование музыкального блога начинается с построения диаграммы прецедентов.

Диаграмма прецедентов – это визуальная модель, описывающая функциональность системы через взаимодействие различных типов пользователей с определёнными сценариями. Для разработки данной модели был использован универсальный язык моделирования UML.

UML является стандартом графического описания для объектного моделирования в разработке программного обеспечения, а также для визуализации бизнес-процессов и системных структур.

UML – это язык «чертежей» для программ, как в строительстве есть чертежи дома.

На рисунке 1 представлена диаграмма прецедентов, которая иллюстрирует ключевые действия Гостя, Пользователя и Администратора в рамках музыкальной платформы:

- Гость имеет ограниченный доступ к системе: он может осуществлять поиск треков и прослушивать их. Также для гостя предусмотрена возможность регистрации в системе для получения расширенных прав.

- Пользователь (зарегистрированный участник) обладает более широким функционалом. Помимо поиска и прослушивания музыки, он может авторизоваться, ставить оценки композициям, а также полноценно управлять своим контентом: загружать собственные треки, редактировать их данные или удалять из своего профиля.

- Администратор выполняет функции контроля и модерации. Он проходит авторизацию для доступа к инструментам управления платформой. В его полномочия входит глобальное управление треками (включая их удаление) и управление учетными записями пользователей (включая удаление профилей нарушителей).

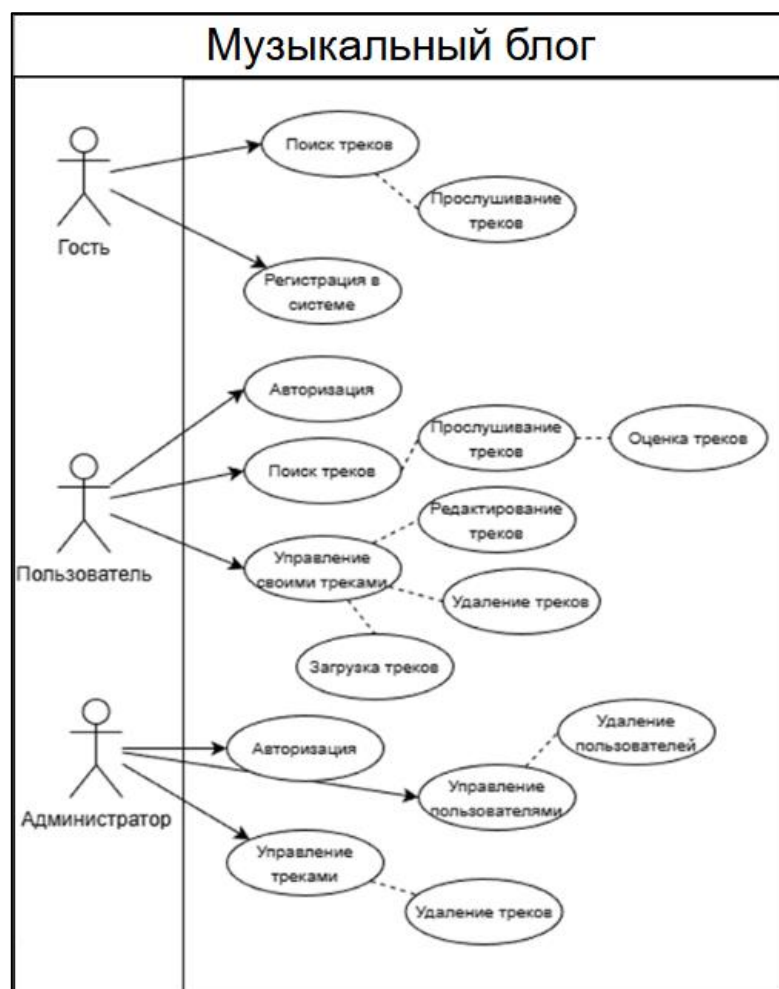


Рисунок 1 – Диаграмма прецедентов

Для детального проектирования логики работы музыкального блога была разработана диаграмма последовательности. В отличие от диаграммы прецедентов, она позволяет визуализировать не просто функции, а временную последовательность обмена сообщениями между пользователями и компонентами музыкального блога.

На рисунке 2 представлена диаграмма, отражающая основные сценарии взаимодействия пользователя, музыкального блога и администратора. Процесс авторизации и поиска: взаимодействие начинается с запроса пользователя на авторизацию. Музыкальный блог проверяет данные в базе и возвращает подтверждение доступа. После этого пользователь может инициировать поиск музыкальных треков по заданным фильтрам (жанр, автор). Музыкальный блог обрабатывает запрос и возвращает актуальный список композиций, доступных для прослушивания и оценки.

Публикация контента: зарегистрированный пользователь имеет возможность загрузить собственный аудиофайл. При этом он передает системе пакет данных: сам файл, название, жанр и описание. Музыкальный блог фиксирует получение данных, присваивает треку статус «На модерации» и отправляет пользователю уведомление об успешной загрузке.

Модерация и управление: администратор взаимодействует с Музыкальным блогом через админ-аккаунт. Он запрашивает список новых треков, ожидающих проверки. Музыкальный блог выводит соответствующие записи. Администратор анализирует контент и принимает решение о публикации или блокировке. После подтверждения администратором Музыкальный блог изменяет статус трека на «Опубликован», делая его видимым для всех пользователей и гостей блога.

Система оценивания: при прослушивании трека пользователь отправляет запрос на выставление оценки. Музыкальный блог принимает числовое значение, выполняет внутренний пересчет среднего рейтинга композиции и обновляет информацию на странице трека в режиме реального времени.

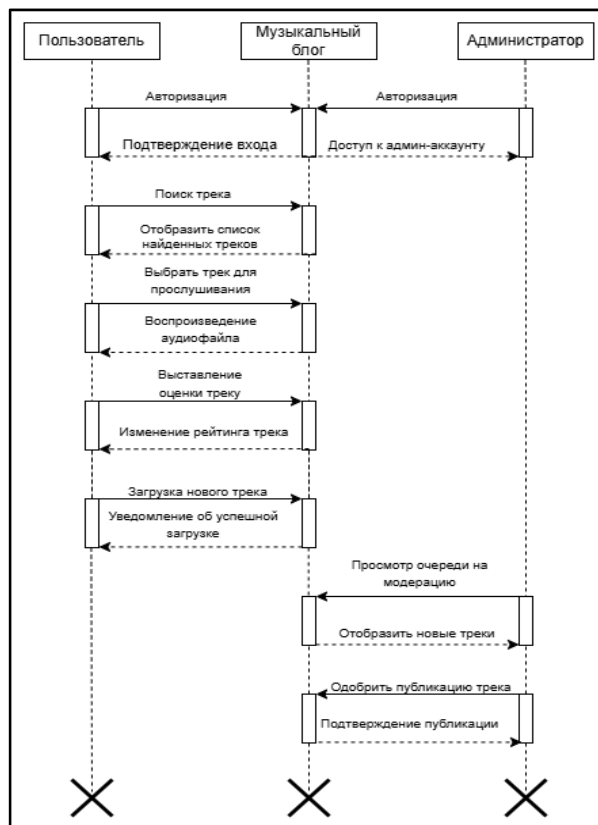


Рисунок 2 – Диаграмма взаимодействий

Для моделирования динамических аспектов и бизнес-логики системы была разработана диаграмма деятельности (Activity Diagram). Она визуализирует конкретные алгоритмы действий и переходы состояний в процессе работы музыкального блога. На рисунке 3 представлена диаграмма деятельности, разделенная на три зоны ответственности (swimlanes): Гость / пользователь, музыкальный блог и администратор.

Описание логики процессов:

- Начальный этап: процесс начинается с открытия сайта музыкального блога и просмотра списка доступных треков.

- Разветвление по ролям: на этапе проверки авторизации алгоритм разделяется:

- Если пользователь не авторизован (Гость), ему доступно только прослушивание композиций.

- Авторизованный Пользователь может не только прослушивать, но и оценивать треки.

- Обработка данных системой: при выставлении оценки музыкальный блог выполняет автоматическое сохранение данных и пересчет общего рейтинга трека в режиме реального времени.

- Публикация контента: при инициировании процесса добавления нового трека пользователем, музыкальный блог выполняет его предварительное сохранение. После этого управление передается Администратору для проведения проверки на корректность и соответствие правилам блога.

- Модерация: по результатам проверки принимается логическое решение:

- При положительном результате (трек корректен) Музыкальный блог осуществляет его финальную публикацию.

- В случае нарушения правил (трек некорректен) происходит удаление записи из системы.

Завершается процесс достижением финального узла после того, как контент был обработан или пользователь завершил сессию прослушивания.

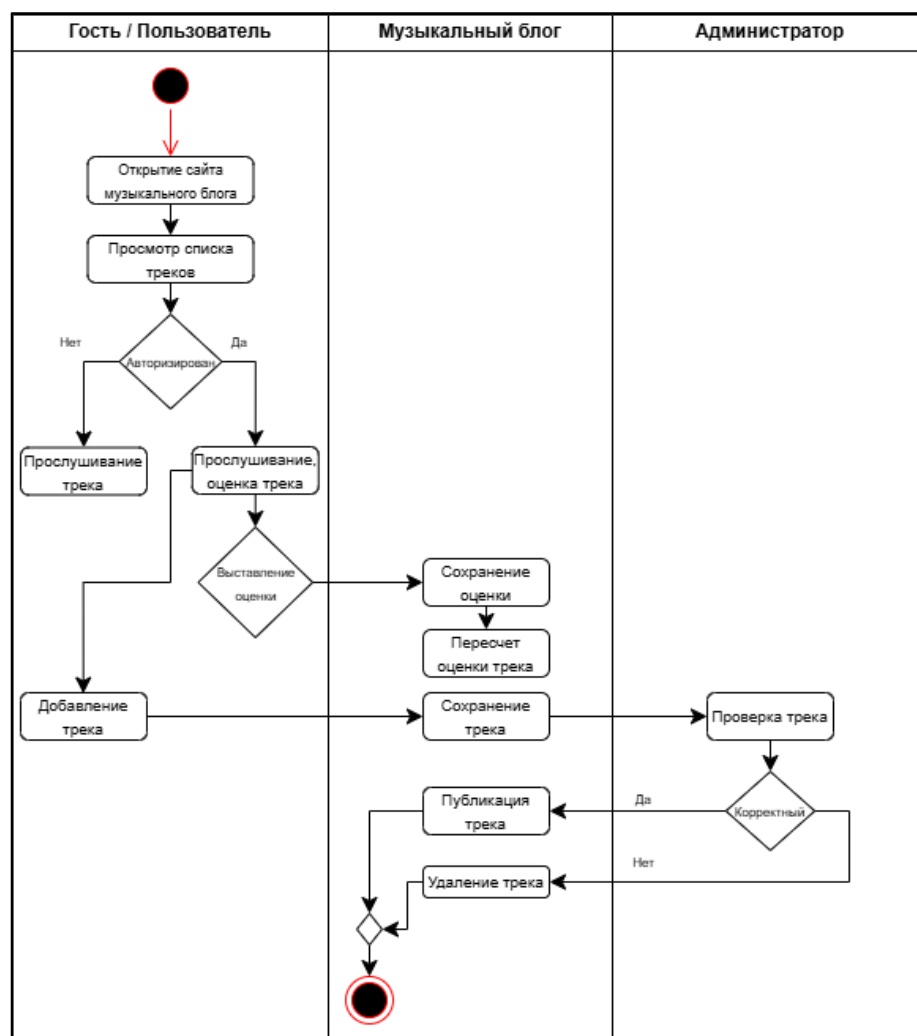


Рисунок 3 – Диаграмма деятельности

4.2 Структурные схемы музыкального блога

Для анализа архитектуры системы на функциональном уровне была построена контекстная диаграмма в нотации IDEF0. Этот метод позволяет представить систему как единый функциональный блок, фокусируясь на том, какие ресурсы она потребляет и какие результаты выдает, не углубляясь в детали реализации. На рисунке 4 представлена диаграмма A0, описывающая процесс «Функционирование музыкального блога»:

— Функциональный блок: основная задача музыкального блога-автоматизация процессов публикации, хранения и оценки музыкальных

произведений. Музыкальный блог выступает связующим звеном между независимыми авторами и слушателями.

— Входные данные: то, что подлежит обработке. К ним относятся исходные музыкальные файлы (MP3, WAV), метаданные (название, жанр, автор), а также данные от зарегистрированных пользователей (оценки и баллы).

— Управление: правила и ограничения, согласно которым работает блог. Сюда входят «Правила модерации контента», технические требования к аудиофайлам и регламент регистрации пользователей.

— Механизмы: ресурсы, необходимые для выполнения процесса. В данной системе механизмами являются Администратор (контроль и модерация), Зарегистрированные пользователи (создатели контента).

— Выходные результаты: итоговый продукт деятельности. На выходе мы получаем публично доступный аудио контент, сформированный рейтинг популярности треков и базу данных независимых исполнителей.

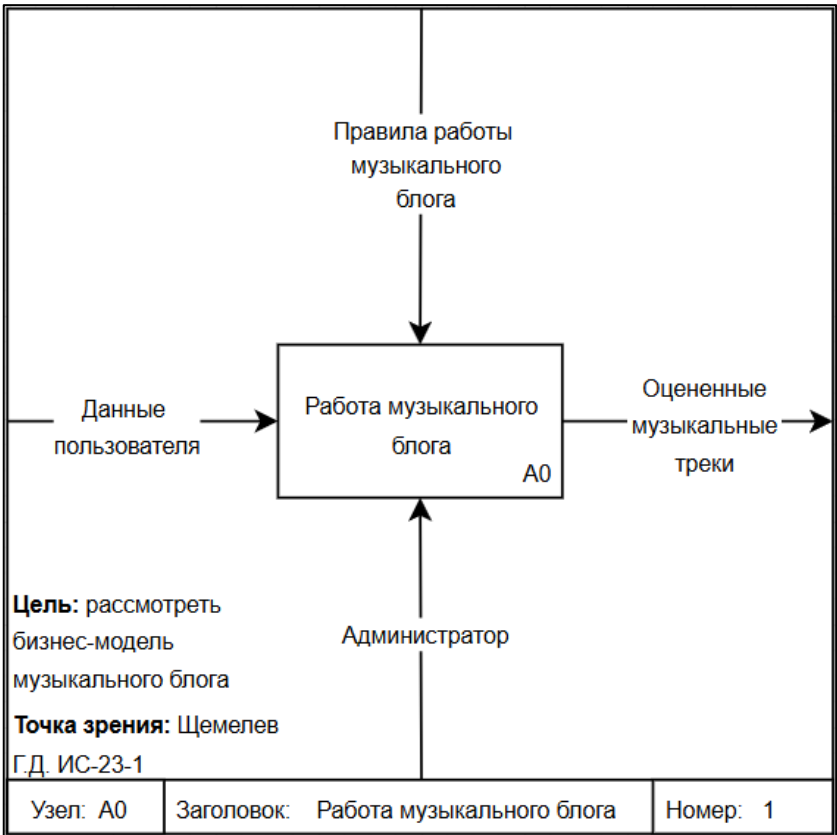


Рисунок 4 – Контекстная диаграмма A0

Для детального анализа внутренней структуры системы была проведена декомпозиция основного процесса «Функционирование музыкального блога». Диаграмма уровня A1 раскрывает логику взаимодействия внутренних модулей, которые на контекстной диаграмме были представлены единым блоком.

На рисунке 5 представлена декомпозиция, состоящая из трех функциональных блоков, расположенных в порядке их выполнения:

- A1 регистрация и авторизация пользователей.

Данный блок является «входными воротами» в систему. Здесь происходит обработка первичных данных пользователя (логин, пароль) и их сверка с базой данных. Выходом блока является «Авторизованный профиль». Это важная интерфейсная дуга, которая направляется в верхние грани последующих блоков (A2 и A3), выступая в роли управления: без подтвержденного профиля система не допускает пользователя к функциям загрузки или оценки контента.

- A2: загрузка и модерация музыкального контента.

В этом блоке реализуется основной бизнес-процесс создания ценности блога. Авторы загружают аудиофайлы и метаданные (название, жанр). Механизмом здесь выступает не только программная часть, но и администратор, который проверяет контент на соответствие правилам. Выходом является «Опубликованный музыкальный трек», который одновременно уходит как готовый результат во внешнюю среду и поступает на вход следующего блока (A3).

- A3: прослушивание и оценка треков.

Завершающий функциональный блок, отвечающий за взаимодействие аудитории с контентом. На вход поступают опубликованные треки из предыдущего этапа. Механизмом является пользователь (слушатель) и встроенный медиаплеер.

Итоговым выходом всей диаграммы здесь становятся «Оценка», которая является результатом коллективной оценки произведений пользователями.

Все блоки объединены общим управлением и общим механизмом, что подчеркивает целостность проектируемого веб-ресурса.

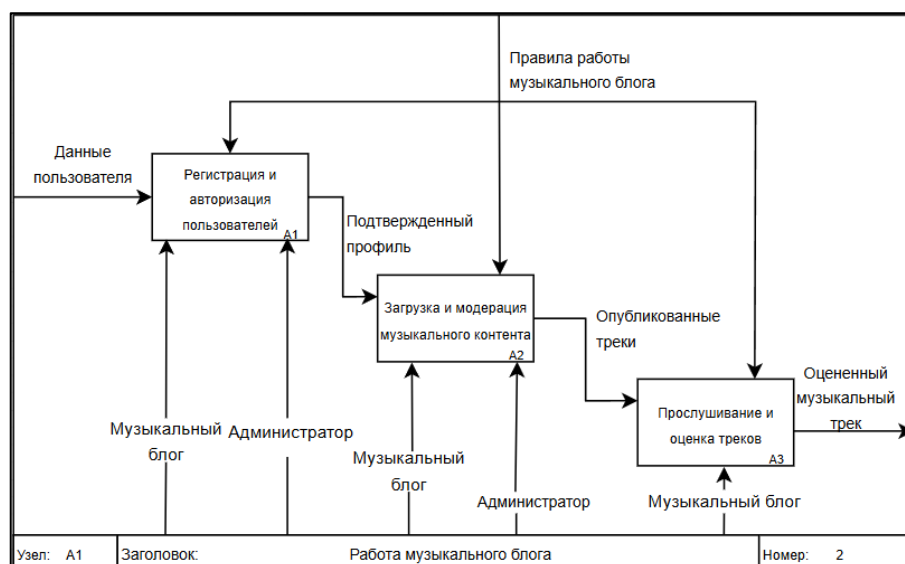


Рисунок 4 – Контекстная диаграмма A1

На рисунке 5 представлена диаграмма классов, которая закладывает программную основу для реализации серверной части системы и написания исполняемого кода. Она демонстрирует, как организованы объекты внутри блога и каким образом реализованы связи между авторами, слушателями и цифровыми произведениями.

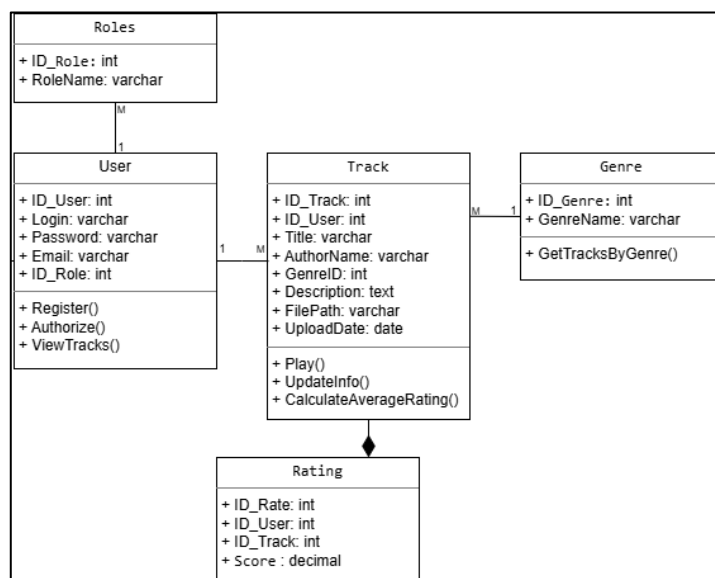


Рисунок 5 – Диаграмма классов

На рисунке 6 представлена диаграмма потоков данных (DFD), которая детально визуализирует маршрутизацию информационных потоков внутри

разрабатываемого музыкального блога. Данная схема позволяет проследить логику перемещения данных от внешних источников (пользователей и администраторов) к процессам обработки и далее к местам их долговременного хранения.

В архитектуре системы четко выделены три хранилища, обеспечивающих структурированное хранение информации:

1. Хранилище пользователи: аккумулирует персональные данные и учетные записи, поступающие на этапе регистрации и авторизации. Это обеспечивает безопасность входа в систему.

2. Хранилище музыкальные треки: служит центральным хранилищем контента. Схема демонстрирует цикличное обращение к этому накопителю: сначала сюда поступают новые файлы при загрузке, затем данные обновляются после модерации администратором, и, наконец, извлекаются системой при запросе пользователя на поиск и прослушивание.

3. Хранилище оценки: специализированный накопитель, фиксирующий результаты пользовательской активности (выставленные рейтинги), что позволяет отделить статистику от основного контента.

Такая организация потоков данных обеспечивает логическую связность процессов и гарантирует, что каждое действие пользователя (будь то загрузка трека или его оценка) фиксируется в соответствующей таблице базы данных.

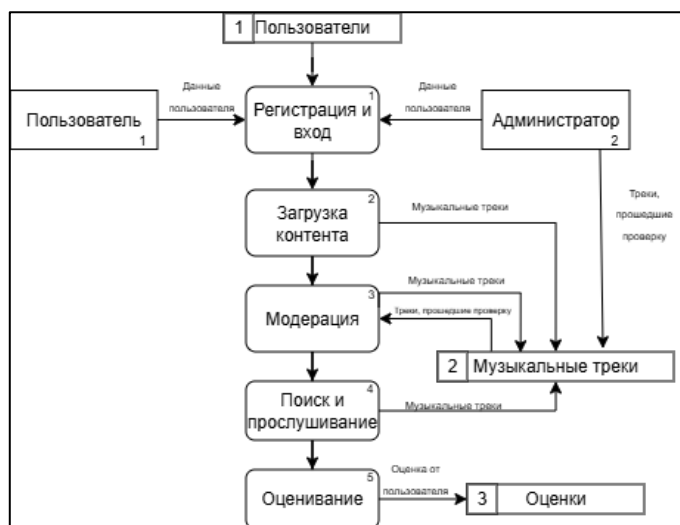


Рисунок 6 – Диаграмма потоков данных

4.3 Проектирование базы данных

На рисунке 7 представлена инфологическая модель музыкального блога. Она отражает семантику предметной области и фиксирует ключевые бизнес-сущности системы:

- Пользователи и Роли: сущность «Роли» позволяет гибко управлять правами доступа, разделяя обычных пользователей, авторов и администраторов.
- Треки и Жанры: каждое музыкальное произведение классифицируется по жанру, что обеспечивает удобную навигацию и поиск для конечного потребителя.
- Статусы: отражают жизненный цикл контента (от черновика и модерации до публикации).
- Оценки: являются связующим звеном в процессе взаимодействия слушателя с контентом, позволяя формировать социальный рейтинг произведений.

Данная модель построена на принципах исключения избыточности и обеспечивает логическую целостность информации на концептуальном уровне.

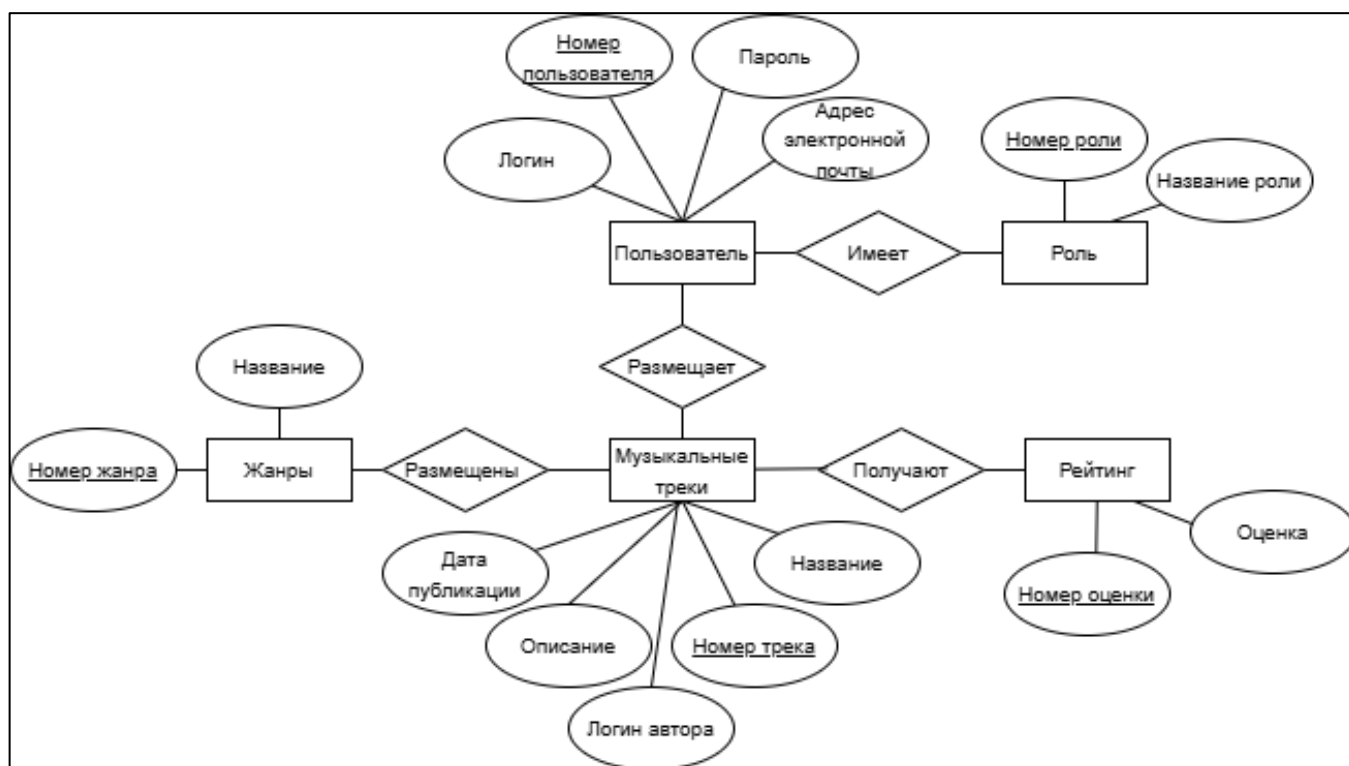


Рисунок 7 – Инфологическая диаграмма

На рисунке 8 представлена даталогическая модель (ER-диаграмма) базы данных.

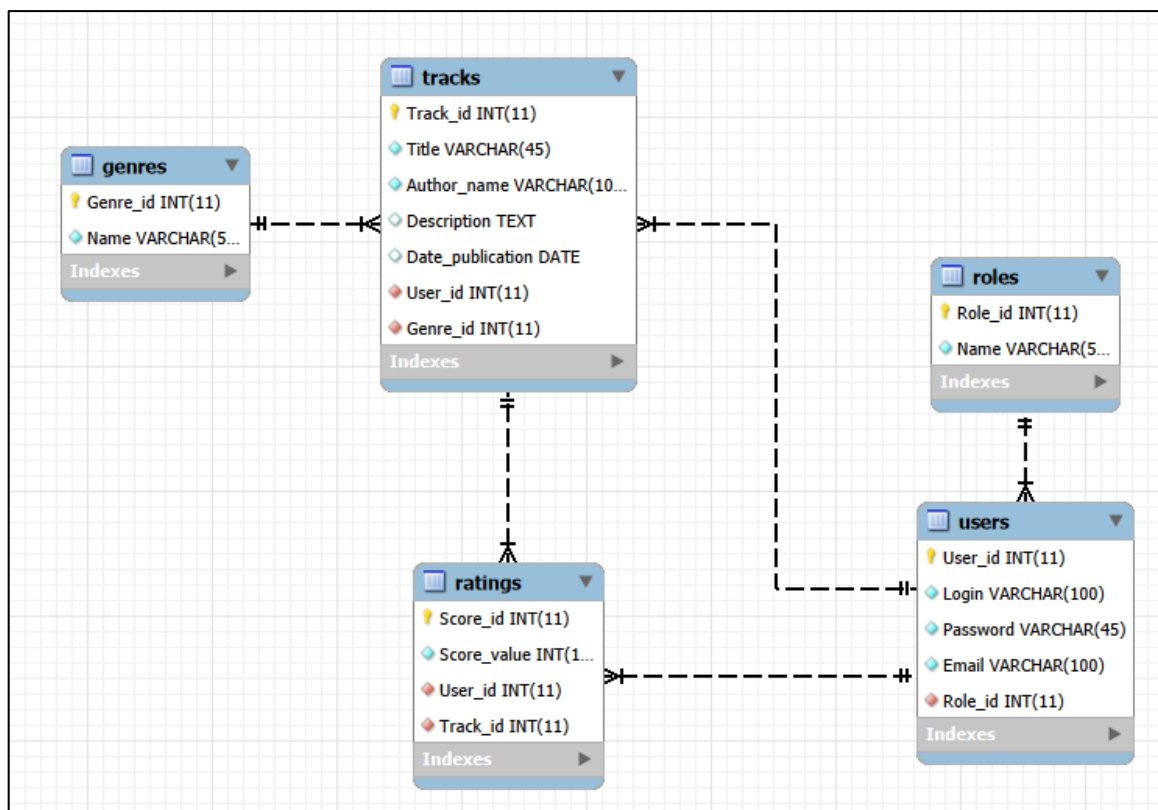


Рисунок 8 – ER диаграмма

Модель реализована в реляционном виде и включает 6 нормализованных таблиц (таблица 5). Все таблицы находятся в третьей нормальной форме (3НФ), что гарантирует устранение транзитивных зависимостей и минимизацию избыточности данных.

Приведение к 3НФ выполнялось последовательно. Сначала была обеспечена первая нормальная форма (1НФ): все атрибуты атомарны, каждая запись уникально идентифицируется первичным ключом. Затем достигнута вторая нормальная форма (2НФ): устранены частичные зависимости неключевых атрибутов от части составного ключа. На заключительном этапе реализована третья нормальная форма (3НФ): полностью исключены транзитивные зависимости, при которых неключевой атрибут зависит от другого неключевого атрибута, а не от первичного ключа.

В результате все неключевые атрибуты в каждой из шести таблиц функционально полно зависят только от первичного ключа. Это устраняет дублирование данных, аномалии обновления и вставки, обеспечивая целостность и непротиворечивость модели.

Таблица 5 – Название и описание таблиц БД

Таблица	Описание
Users	Хранит данные учетных записей
Tracks	Основная таблица системы
Ratings	Хранит оценки пользователей
Roles	Таблица ролей
Genres	Таблица жанров

1. Users (таблица 6): хранит данные учетных записей. Поле `role_id` является внешним ключом к таблице Roles.

Таблица 6 – Таблица Users

Поле	Тип данных	Описание
User_id	int	Номер пользователя (PK)
Login	varchar (100)	Имя пользователя
Password	varchar (45)	Пароль пользователя
Email	varchar (100)	Почта пользователя
Role_id	int	Номер роли пользователя (FK)

2. Tracks (таблица 7): основная таблица системы. Содержит ссылки на автора (`author_id`), жанр (`genre_id`) и текущий статус (`status_id`). Также хранит путь к файлу (`file_path`) и описание.

Таблица 7 – Таблица Tracks

Поле	Тип данных	Описание
Track_id	int	Номер трека (PK)
Title	varchar (45)	Название трека
Author_name	varchar (100)	Имя автора
Description	Text	Описание трека
Date_publication	Date	Дата публикации трека

Продолжение таблицы 7

User_id	int	Номер пользователя (FK)
Genre_id	int	Номер жанра трека (FK)

3. Ratings (таблица 8): реализует связь «многие-ко-многим» между пользователями и треками, позволяя хранить уникальное значение балла (score) для каждой пары «пользователь-трек».

Таблица 8 – Таблица Ratings

Поле	Тип данных	Описание
Score_id	int	Номер оценки (PK)
Score_value	int	Численная оценка
User_id	int	Номер пользователя (FK)

4. Справочники (Roles, Genres) (таблицы 9 и 10): состоят из уникального идентификатора и наименования. Использование отдельных таблиц вместо перечисляемых типов позволяет изменять состав категорий без вмешательства в структуру базы данных.

Таблица 9 – Таблица Roles

Поле	Тип данных	Описание
Role_id	int	Номер роли (PK)
Name	varchar (50)	Название трека

Таблица 10 – Таблица Genres

Поле	Тип данных	Описание
Genre_id	int	Номер жанра (PK)
Name	varchar (50)	Название жанра

4.4 Проектирование интерфейса

Для реализации прототипов пользовательского интерфейса был использован онлайн сервис Figma.

В результате проектирования интерфейса музыкального блога были спроектированы прототипы четырёх страниц.

На рисунке 9 представлен прототип «Главная страница». Прототип демонстрирует основной экран взаимодействия пользователя с контентом.

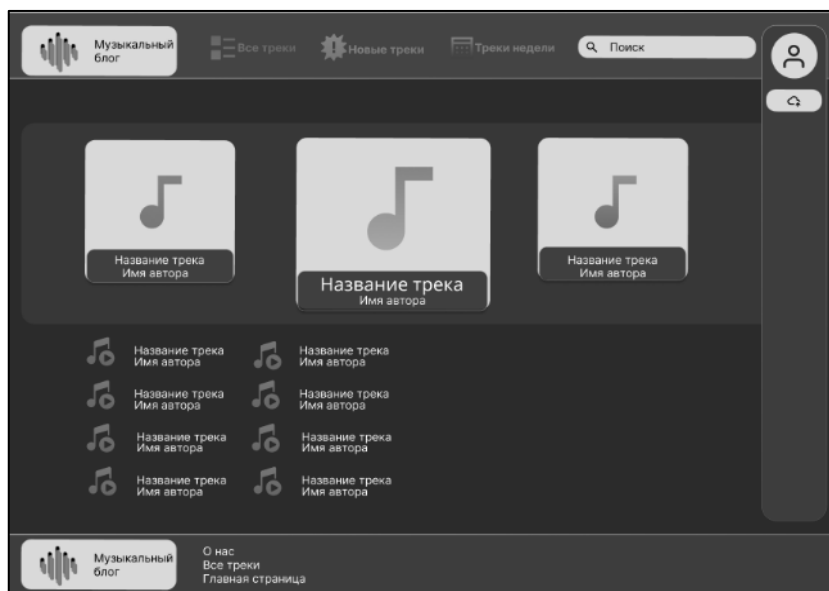


Рисунок 9 – Прототип главной страницы

На рисунке 10 представлен прототип «Регистрация», который отражает, что гость будет вводить при регистрации своего профиля.

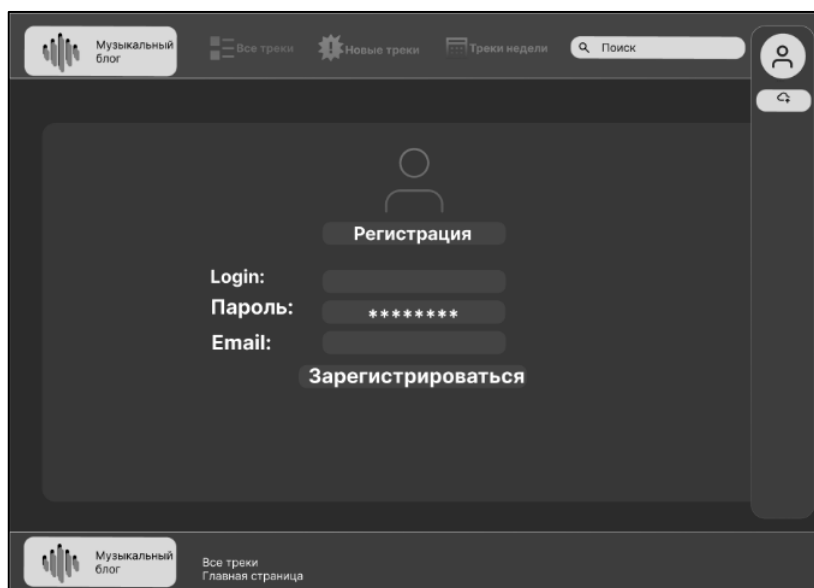


Рисунок 10 – Прототип страницы регистрации

На рисунке 11 изображён прототип «Авторизация», содержащий форму для регистрации и входа в систему.

Музыкальный блог

Все треки

Новые треки

Треки недели

Поиск

Авторизация

Login:

Пароль: *****

Email:

ВОЙТИ

Нет аккаунта?

Музыкальный блог

Все треки

Главная страница

Рисунок 11 – Прототип страницы авторизации

5 Разработка музыкального блога

5.1 Разработка интерфейса музыкального блога

Для разработки интерфейса музыкального блога были использованы HTML, CSS и JavaScript. HTML обеспечивает структуру страниц, CSS отвечает за визуальное оформление, а JavaScript добавляет интерактивность элементам интерфейса, таким как поиск по трекам и фильтрация по жанрам.

Первым этапом была создана базовая структура страниц с использованием шаблонизатора Django. Все страницы наследуются от базового шаблона `base.html`, который содержит общие элементы: шапку с логотипом, навигационное меню, поисковую строку и панель пользователя, а также подвал сайта.

На рисунке 12 представлен код базового шаблона (`base.html`), который обеспечивает единый стиль для всех страниц музыкального блога.

```
<body>
  <div class="desktop">
    <header class="header">
      

      <div class="logo-wrapper div">
        <div class="logo-icons">
          
          
          
        </div>
        <span class="text-wrapper-4">Музыкальный<br>блог</span>
      </div>

      <nav class="main-nav">
        <a href="/tracks/" class="nav-link">
          
          <span class="text-wrapper">Все треки</span>
        </a>
        <a href="/tracks/new/" class="nav-link">
          
          <span class="text-wrapper-2">Новые треки</span>
        </a>
        <a href="/tracks/week/" class="nav-link">
          
          <span class="text-wrapper-3">Треки недели</span>
        </a>
      </nav>
    </div>
  </div>
```

Рисунок 12 – Фрагмент листинга кода `base.html`

На рисунке 13 представлена стилизация шапки музыкального блога, включающая градиентные кнопки, адаптивное меню и пользовательскую панель.

```

11     html, body {
12         width: 100%;
13         height: 100%;
14         overflow-x: hidden;
15     }
16
17     body {
18         background-color: #262534;
19         font-family: 'Inter', sans-serif;
20         color: white;
21         min-height: 100vh;
22     }
23

```

Рисунок 13 – Фрагмент листинга кода style.css

На рисунке 14 представлен итоговый результат – главная страница музыкального блога с отображением треков, навигацией и поиском.

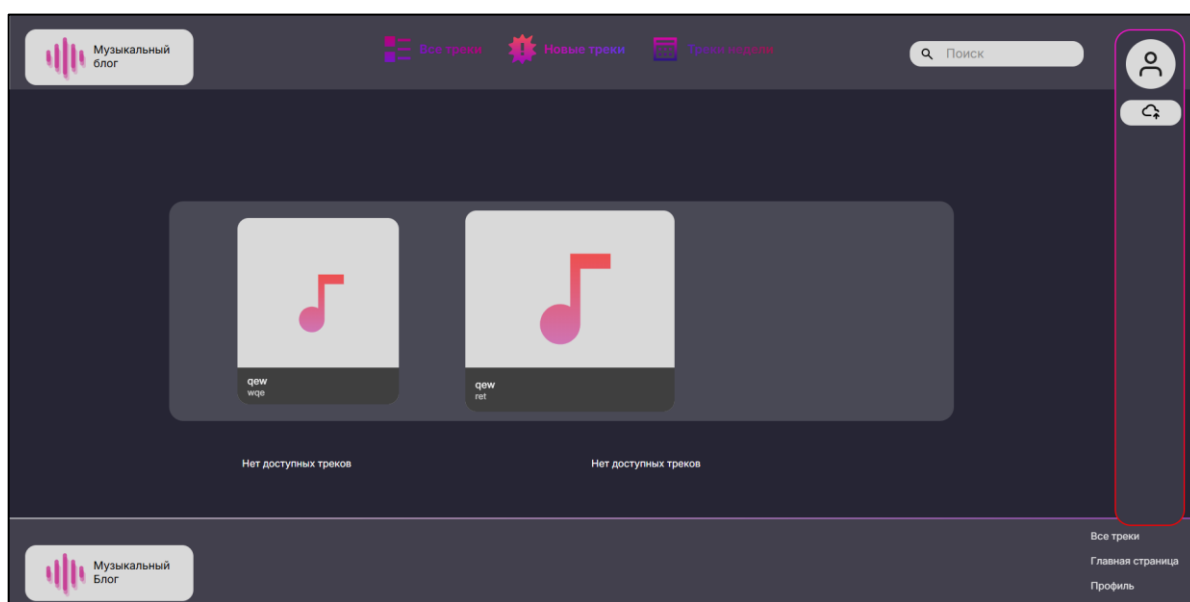


Рисунок 14 – Главная страница музыкального блога

5.2 Разработка базы данных музыкального блога

В Django используется встроенная система ORM, которая позволяет работать с базой данных через Python-объекты. Модели Django описывают структуру таблиц, а методы моделей выполняют операции с данными (добавление, чтение, обновление, удаление) без написания SQL-запросов.

Разработка базы данных начинается с создания моделей в файле `models.py` приложения `tracks`. На основе этих моделей Django автоматически генерирует миграции – файлы с описанием изменений структуры таблиц. Миграции позволяют создавать и обновлять базу данных, а также отслеживать историю всех изменений.

Для подключения к базе данных в MySQL был настроен `settings.py` (рисунок 15).

```
12  ▾ DATABASES = {
13      'default': {
14          'ENGINE': 'django.db.backends.mysql',
15          'NAME': os.getenv('DB_NAME', 'music_blog'),
16          'USER': os.getenv('DB_USER', 'root'),
17          'PASSWORD': os.getenv('DB_PASSWORD', ''),
18          'HOST': os.getenv('DB_HOST', 'localhost'),
19          'PORT': os.getenv('DB_PORT', '3306'),
20      }
21  }
```

Рисунок 15 – Подключение к БД MySQL

Для создания миграций используется команда ``python manage.py makemigrations``. После создания всех миграций они применяются к базе данных командой ``python manage.py migrate``.

На рисунках 15-19 представлены модели и миграции для создания базы данных музыкального блога.

```
migrations.CreateModel(
    name='Role',
    fields=[
        ('role_id', models.AutoField(primary_key=True, serialize=False)),
        ('name', models.CharField(max_length=50)),
    ],
    options={
        'db_table': 'Roles',
    },
),
```

Рисунок 16 – Миграция ролей

```
migrations.CreateModel(
    name='User',
    fields=[
        ('user_id', models.AutoField(primary_key=True, serialize=False)),
        ('login', models.CharField(max_length=100, unique=True)),
        ('password', models.CharField(max_length=45)),
        ('email', models.EmailField(max_length=100, unique=True)),
        ('role', models.ForeignKey(db_column='Role_id', on_delete=django.db.models.deletion.RESTRICT, to='tracks
    ],
    options={
        'db_table': 'Users',
    },
),
```

Рисунок 17 – Миграция пользователей

```
migrations.CreateModel(
    name='Genre',
    fields=[
        ('genre_id', models.AutoField(primary_key=True, serialize=False)),
        ('name', models.CharField(max_length=50)),
    ],
    options={
        'db_table': 'Genres',
    },
),
```

Рисунок 18 – Миграция жанров

```
name='Track',
fields=[
    ('track_id', models.AutoField(primary_key=True, serialize=False)),
    ('title', models.CharField(max_length=45)),
    ('author_name', models.CharField(max_length=100)),
    ('description', models.TextField(blank=True, null=True)),
    ('date_publication', models.DateField(auto_now_add=True)),
    ('audio_file', models.FileField(blank=True, null=True, upload_to='tracks/', verbose_name='Аудиофайл')),
    ('genre', models.ForeignKey(db_column='Genre_id', on_delete=django.db.models.deletion.RESTRICT, to='tracks.genre')),
    ('user', models.ForeignKey(db_column='User_id', on_delete=django.db.models.deletion.CASCADE, to='tracks.user')),
],
options={
    'db_table': 'Tracks',
},
```

Рисунок 19 – Миграция треков

```
name='Rating',
fields=[
    ('score_id', models.AutoField(primary_key=True, serialize=False)),
    ('score_value', models.IntegerField()),
    ('track', models.ForeignKey(db_column='Track_id', on_delete=django.db.models.deletion.CASCADE, to='tracks.track')),
    ('user', models.ForeignKey(db_column='User_id', on_delete=django.db.models.deletion.CASCADE, to='tracks.user')),
],
options={
    'db_table': 'Ratings',
    'unique_together': (('user', 'track')),
},
```

Рисунок 20 – Миграция рейтинга

5.3 Разработка музыкального блога

Разработка музыкального блога велась с использованием веб Фреймворка Django, который предоставляет встроенные инструменты для работы с базой данных, аутентификацией пользователей и маршрутизацией.

Настройка подключения к базе данных выполнена в файле `settings.py`. В качестве СУБД используется MySQL, параметры подключения указаны в блоке `DATABASES`.

Модели данных описаны в файле `models.py` приложения `tracks`. Для каждой таблицы создан соответствующий класс модели, определены поля и связи между таблицами. После описания моделей были созданы и применены миграции.

Представления (`views`) содержат бизнес-логику приложения. Каждое представление обрабатывает определённый URL-адрес, выполняет запросы к базе данных и возвращает HTML-страницу.

В музыкальном блоге реализованы представления для:

- отображения главной страницы (`index`);
- отображения списка всех треков (`tracks_list`);
- отображения новых треков за последние 7 дней (`tracks_new`);
- отображения топ10 треков недели по оценкам (`tracks_week`);
- отображения страницы конкретного трека (`track_detail`);
- публикации нового трека (`upload_track`);
- регистрации пользователя (`register`);
- авторизации пользователя (`user_login`);
- выхода из системы (`user_logout`);
- отображения профиля пользователя (`profile`);
- страницы модерации для администратора (`admin_tracks`);
- обработки оценки трека (`rate_track`).

На рисунках 21-23 представлены листинги основных представлений:

- Главной страницы (рисунок 21), где пользователю отображаются треки, навигация по музыкальному блогу.

— Страница публикации трека (рисунок 22, приложение Б), где пользователь может опубликовать свой трек.

— Страница модерации треков (рисунок 23), где администратор может одобрить либо отклонить трек, который загрузил пользователь.

```
{% block content %}
<section class="featured-section rectangle-featured-section">
  <div class="featured-bg rectangle-6"></div>
  {% for track in featured_tracks|slice:":3" %}
  <article class="featured-card {% if forloop.counter == 2 %}featured-card-center{% endif %} rectangle-{% if forloop.counter == 1 %}9{% elif forloop.counter == 2 %}7{% else %}8{% endif %}"
    data-track-id="{{ track.track_id }}">
    
    <div class="featured-card_overlay rectangle-{% if forloop.counter == 2 %}11{% elif forloop.counter == 1 %}12{% else %}10{% endif %}">
      <p class="text-wrapper-26 track-name">{{ track.title }}</p>
      <p class="text-wrapper-30 artist-name">{{ track.author_name }}</p>
    </div>
  </article>
  {% endfor %}
</section>

<section class="tracklist-section">
  <ul class="track-column track-column--left">
    {% for track in featured_tracks|slice:":7" %}
    <li class="track-item" data-track-id="{{ track.track_id }}">
      
      <div class="track-item_info">
        <span class="track-name">{{ track.title }}</span>
        <span class="artist-name">{{ track.author_name }}</span>
      </div>
    </li>
    {% empty %}
    <li class="track-item">Нет доступных треков</li>
    {% endfor %}
  </ul>
</section>
```

Рисунок 21 — Фрагмент листинга кода главной страницы

```
{% block content %}
<div class="auth-page-content">
  <div class="auth-box upload-box">
    <h1 class="auth-title">Публикация трека</h1>

    <form method="POST" action="/upload/" enctype="multipart/form-data" class="auth-form">
      {% csrf_token %}
      <input type="text" class="auth-input" name="track_name" placeholder="Название трека" required>
      <input type="text" class="auth-input" name="artist_name" placeholder="Имя автора" required>

      <select class="auth-input auth-select" name="genre" required>
        <option value="" disabled selected>Выберите жанр</option>
        {% for genre in genres %}
        <option value="{{ genre.genre_id }}">{{ genre.name }}</option>
        {% endfor %}
      </select>

      <textarea class="auth-input auth-textarea" name="description" placeholder="Описание трека" rows="5"></textarea>

      <!-- Поле для загрузки аудиофайла -->
      <div class="file-upload-zone" id="drop-zone">
        <input type="file" name="audio_file" id="file-input" class="hidden-file-input" accept="audio/*" required>
        <label for="file-input" class="file-label">
          
          <span class="upload-text">Выберите файл или перетащите его сюда</span>
          <span class="file-name-display" id="file-name">Файл не выбран</span>
        </label>
      </div>

      <button type="submit" class="auth-submit">Опубликовать</button>
    </form>
  </div>
</div>
```

Рисунок 22 — Фрагмент листинга кода страницы публикации трека

```
{% block content %}
<div style="padding: 160px 40px 50px; max-width: 1200px; margin: 0 auto;">
  <h1 style="color: white; margin-bottom: 30px;"> Модерация треков</h1>

  <!-- Треки на модерации -->
  <h2 style="color: #FF11D3; margin: 30px 0 20px;"> На модерации ({{ pending_tracks|length }})</h2>
  <div class="tracks-list">
    {% for track in pending_tracks %}
    <div class="track-card pending" data-track-id="{{ track.track_id }}">
      <div class="track-info">
        <strong>{{ track.title }}</strong> - {{ track.author_name }}
        <br><small>Автор: {{ track.user.login }} | Жанр: {{ track.genre.name }} | {{ track.date_publication }}</small>
        <br><small class="description">{{ track.description|truncatechars:100 }}</small>
        <br><small><img alt="audio icon" data-bbox="265 178 278 188"/> Аудио: {% if track.audio_file %}<a href="{{ track.audio_file.url }}" target="_blank">Прослушать</a>{% endif %}</small>
      </div>
      <div class="track-actions">
        <textarea id="comment-{{ track.track_id }}" placeholder="Комментарий модератора (опционально)" rows="2" style="width: 300px; margin-bottom: 10px; padding: 8px;"></textarea>
        <div>
          <button class="btn-approve" onclick="moderateTrack('{{ track.track_id }}', 'approve')"> Одобрить</button>
          <button class="btn-reject" onclick="moderateTrack('{{ track.track_id }}', 'reject')"> Отклонить</button>
        </div>
      </div>
    </div>
    {% empty %}
    <p style="color: white;">Нет треков на модерации</p>
    {% endfor %}
  </div>

  <!-- Одобренные треки -->
  <h2 style="color: #4CAF50; margin: 40px 0 20px;"> Одобренные треки ({{ approved_tracks|length }})</h2>
  <div class="tracks-list approved">
    {% for track in approved_tracks %}
    <div class="track-card">
      <div class="track-info">
```

Рисунок 23 — Фрагмент листинга кода страницы модерации треков

Маршрутизация URL настроена в файле `urls.py` приложения `tracks`. Каждый URL-адрес связан с соответствующим представлением. Также в корневом файле `urls.py` проекта настроена обработка статических и медиафайлов.

Шаблоны (templates) построены с использованием системы наследования шаблонов Django. Базовый шаблон `base.html` содержит общую структуру страниц, а дочерние шаблоны (`index.html`, `track.html`, `upload.html`, `profile.html`, `login.html`, `register.html`, `admin_tracks.html`) переопределяют блоки с контентом. В шаблонах реализованы:

- вывод списка треков с помощью циклов;
- условное отображение элементов в зависимости от статуса авторизации и роли пользователя;
- формы для регистрации, авторизации и публикации треков;
- JavaScript-функции для поиска и фильтрации треков по жанру и названию;
- AJAX-запросы для выставления оценок без перезагрузки страницы.
- Статические файлы (CSS, изображения, иконки) размещены в папке `static` и подключаются через тег `{% load static %}`.

— Медиафайлы (аудиофайлы треков) загружаются пользователями через форму публикации и сохраняются в папку media. Настройки для работы с медиафайлами указаны в settings.py и urls.py.

— Система модерации реализована с помощью поля status в модели Track.

Треки, загруженные пользователями, получают статус "pending". Администратор на странице модерации может одобрить (status = "approved") или отклонить (status = "rejected") трек, указав комментарий. Одобренные треки отображаются на всех страницах сайта, а отклонённые — только в профиле пользователя с комментарием модератора.

Система оценок позволяет зарегистрированным пользователям выставять оценки от 1 до 10. Оценки сохраняются в таблице Ratings, средний рейтинг пересчитывается автоматически и отображается на странице трека.

6 Документирование программного продукта

6.1 Руководство пользователя музыкального блога

Для создания учетной записи необходимо перейти на страницу «Регистрация» по ссылке в навигационном меню. На странице регистрации (рисунок 24) требуется заполнить поля:

- Email – действующий адрес электронной почты;
- Логин – уникальное имя пользователя;
- Пароль и подтверждение пароля.

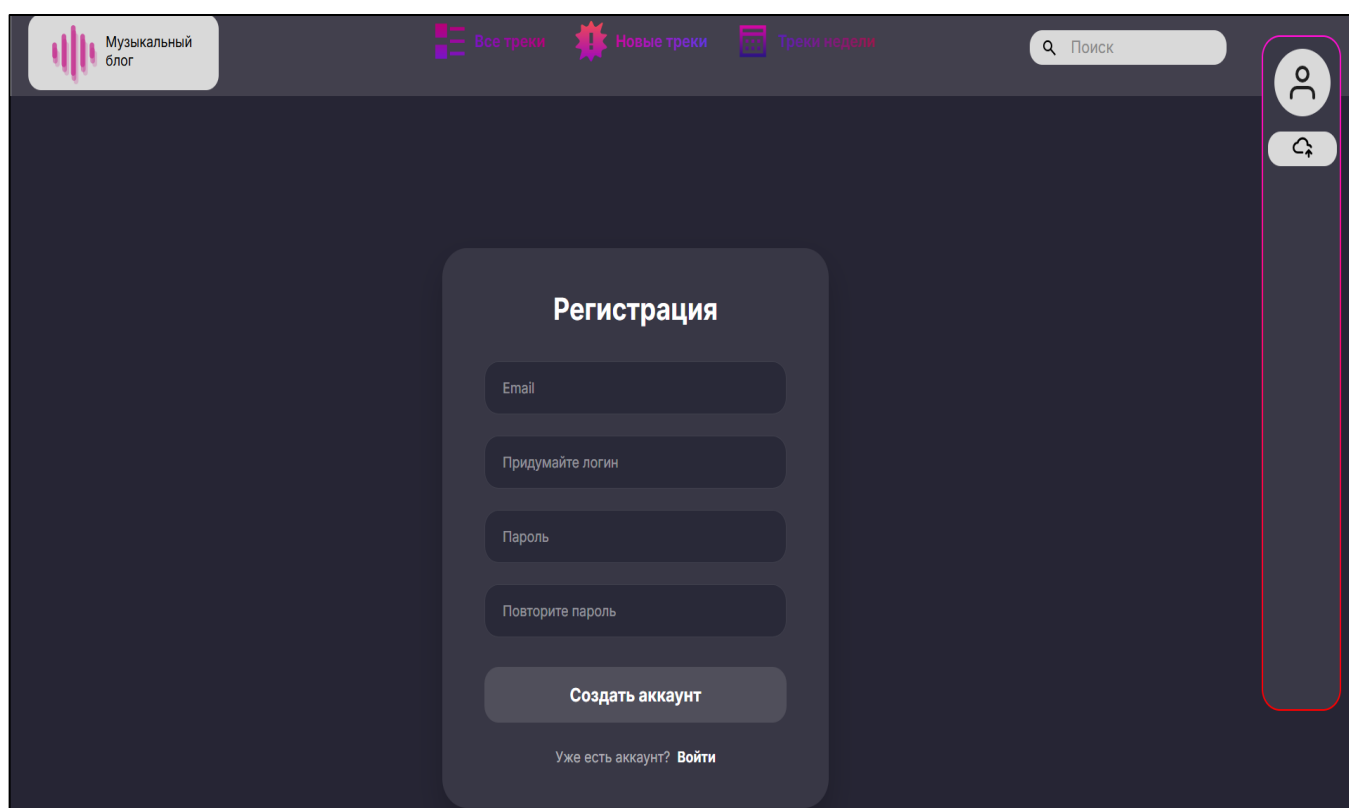


Рисунок 24 – Страница регистрации

После успешной регистрации пользователь перенаправляется на страницу авторизации.

Для входа в систему используется страница «Вход» (рисунок 25). Необходимо ввести логин и пароль, указанные при регистрации. После успешной авторизации пользователь получает доступ к полному функционалу музыкального блога.

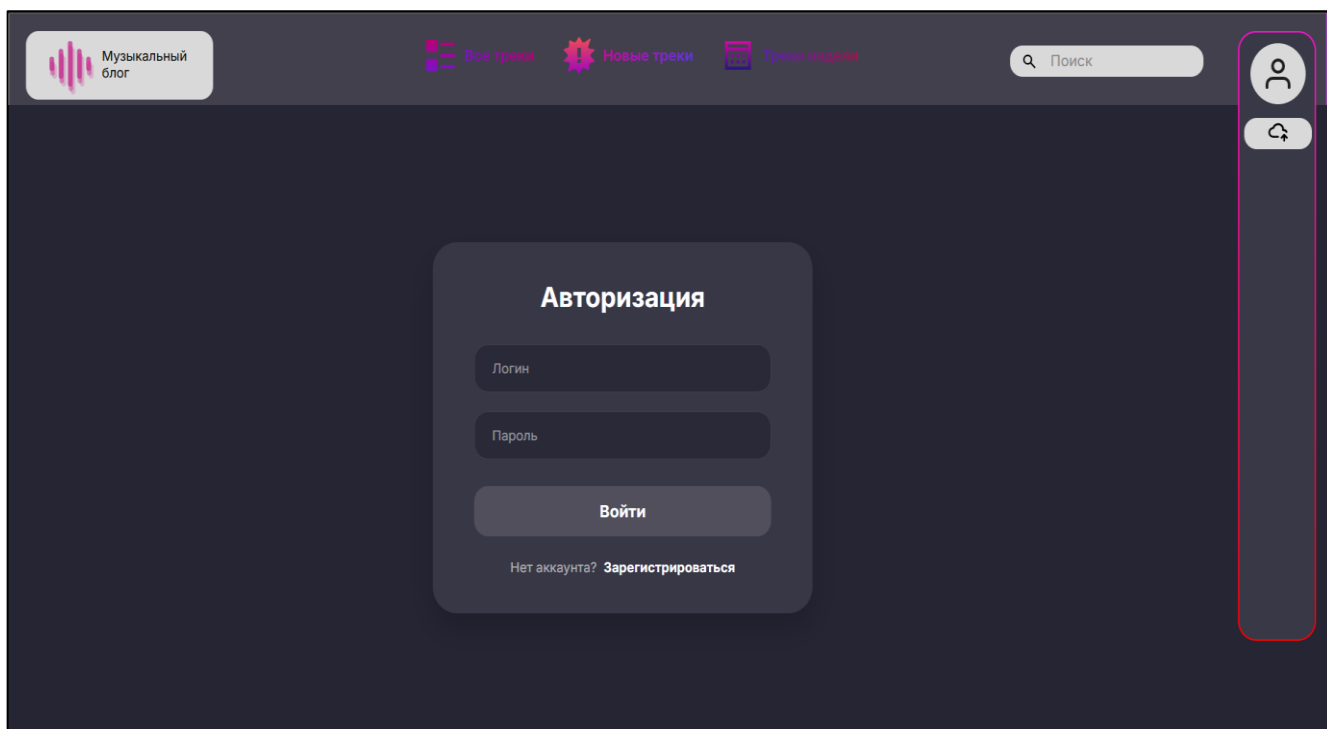


Рисунок 25 – Страница авторизации

На главной странице (рисунок 26) отображаются популярные и последние добавленные треки. Для перехода к странице трека необходимо нажать на карточку трека или на элемент списка.

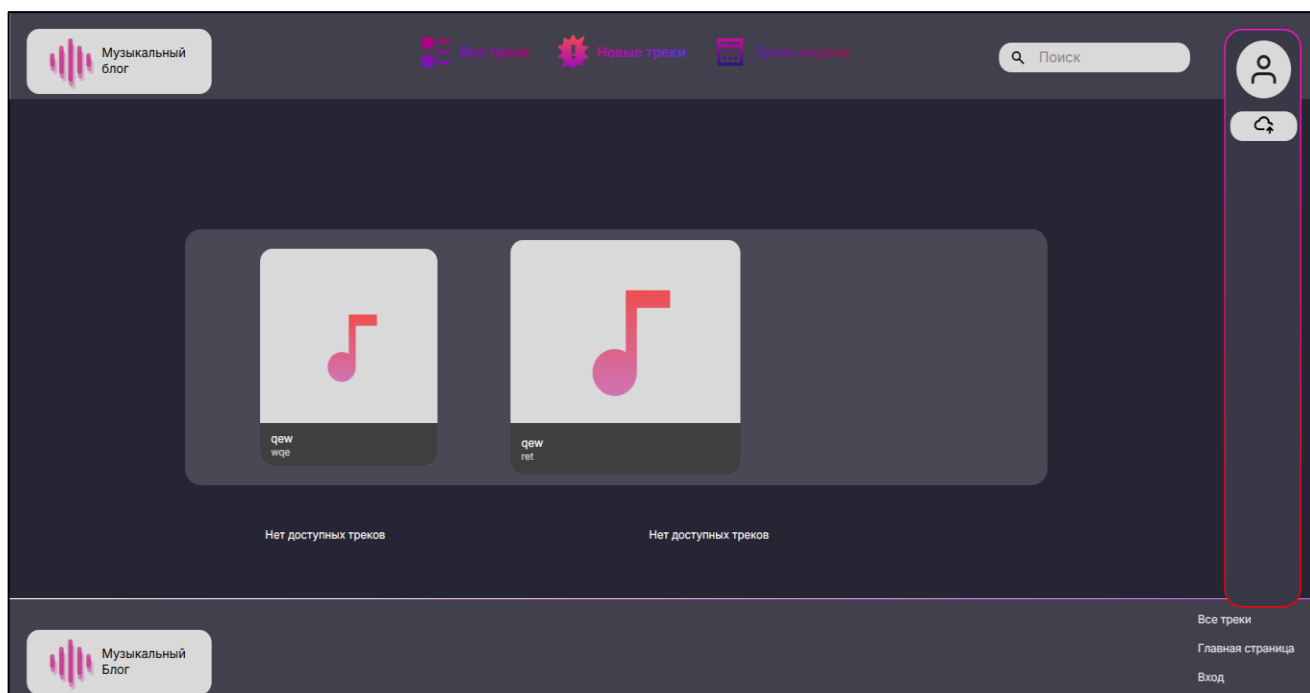


Рисунок 26 – Главная страница

На странице трека (рисунок 27) доступны:

- прослушивание аудиофайла с помощью встроенного плеера;
- просмотр описания, жанра и рейтинга;
- выставление оценки от 1 до 10 (доступно только авторизованным

пользователям).

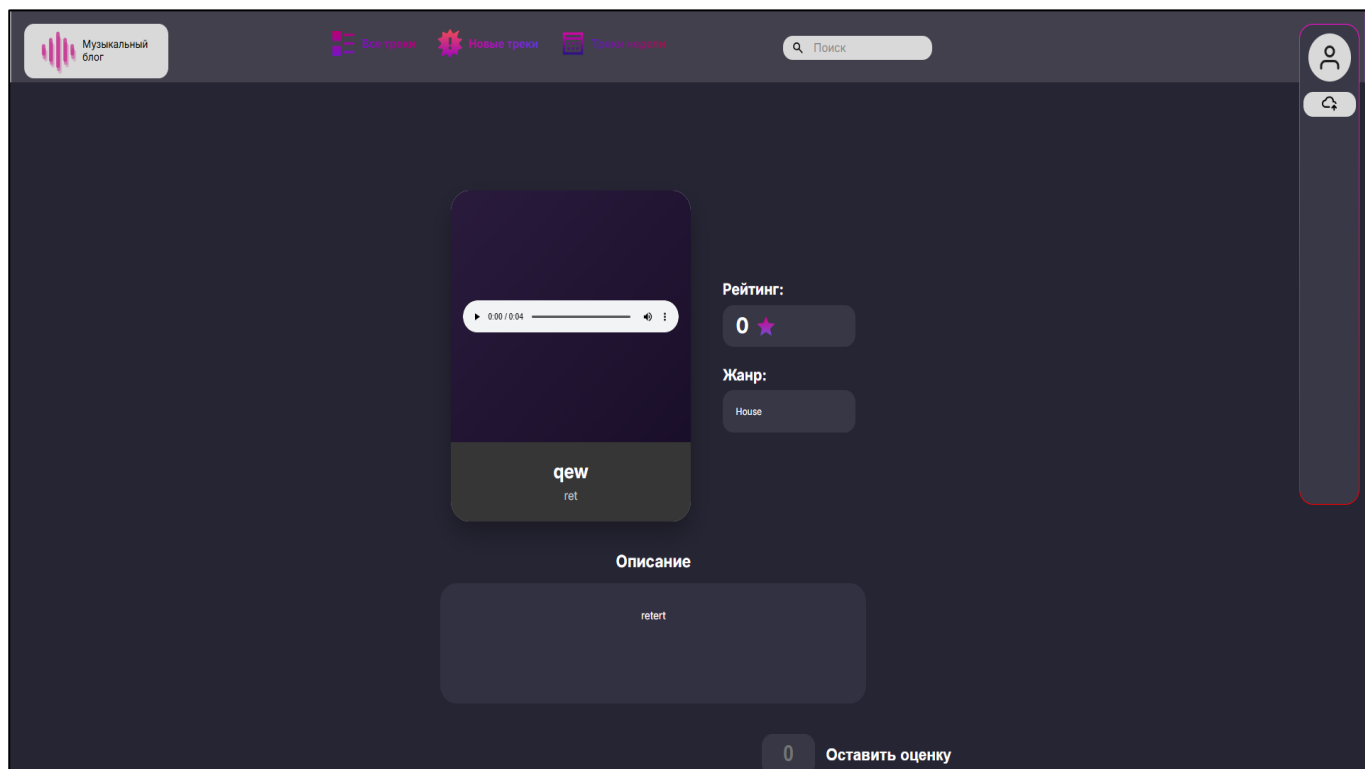


Рисунок 27 – Страница карточки трека

Авторизованные пользователи могут публиковать собственные треки. Для этого необходимо нажать на иконку облака в правой панели или перейти по ссылке «Опубликовать трек» в профиле. На странице публикации (рисунок 28) требуется заполнить:

- название трека;
- имя автора;
- жанр (выбирается из списка);
- описание (необязательно);
- аудиофайл в формате MP3, WAV или других популярных форматах.

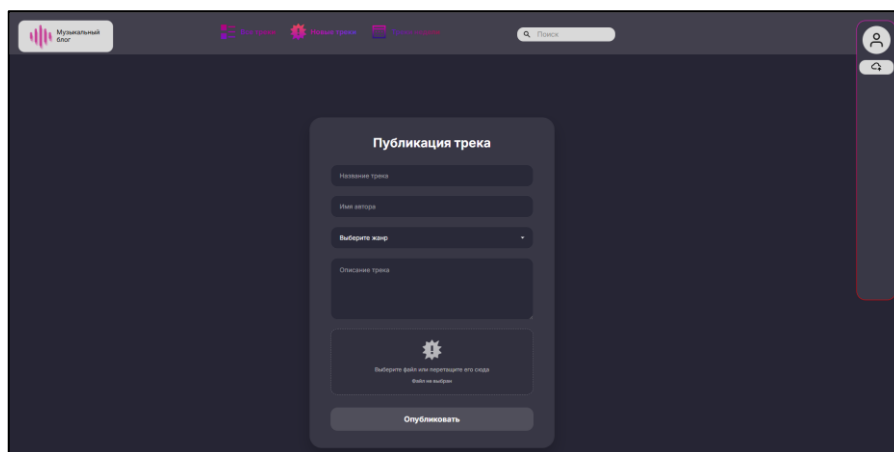


Рисунок 28 – Страница публикации трека

После отправки трек получает статус «На модерации» и становится доступен для проверки администратором.

В личном кабинете (рисунок 29) отображаются:

- информация о пользователе (email, логин);
- список опубликованных треков с указанием статуса:
 1. «На модерации» – трек ожидает проверки;
 2. «Опубликован» – трек одобрен и виден всем пользователям;
 3. «Отклонен» – трек не прошел модерацию, причина указана в комментарии.

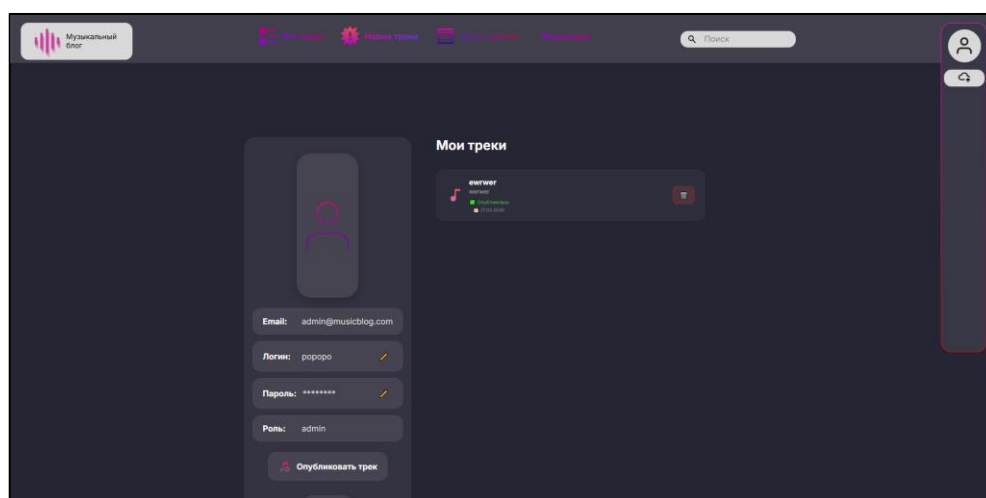


Рисунок 29 – Страница личного кабинета

На страницах «Все треки» и «Новые треки» (рисунок 30 и 31) доступны:

- поиск по названию трека и имени автора в реальном времени;
- фильтрация по жанру с помощью выпадающего списка.

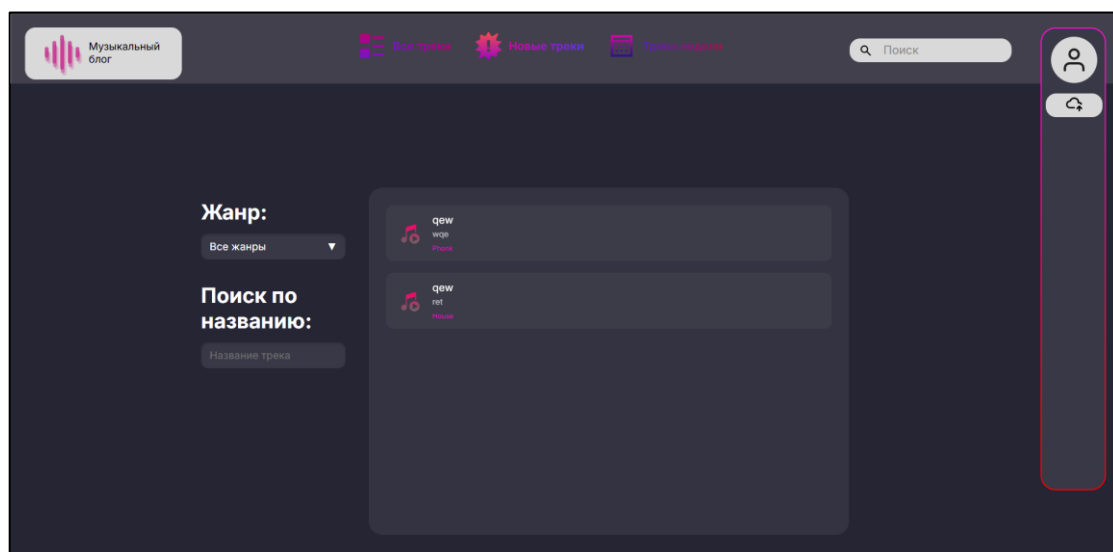


Рисунок 30 – Страница «все треки»

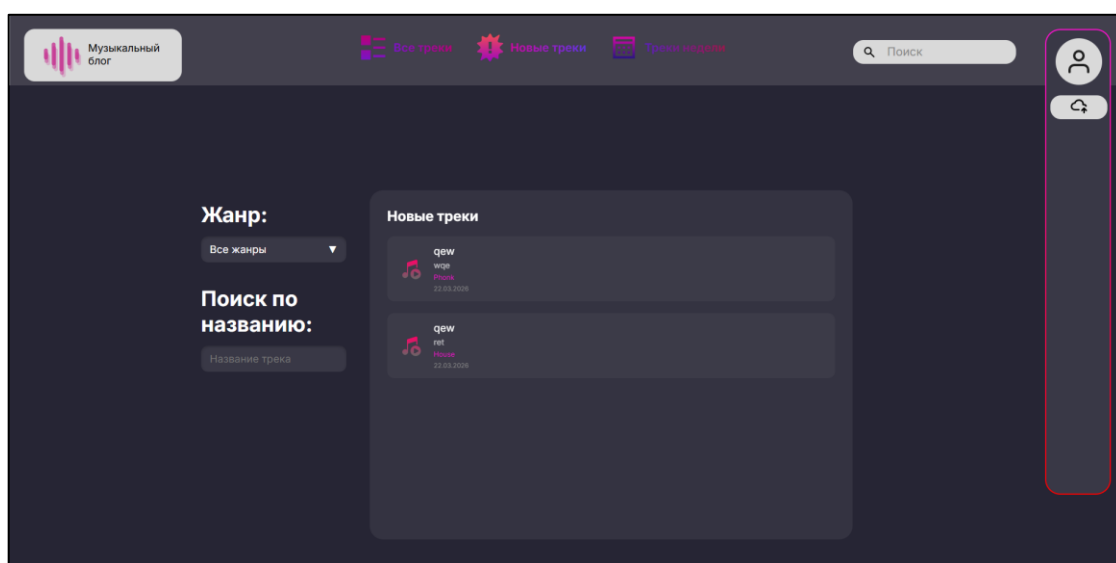


Рисунок 31 – Страница «новые треки»

Администратор имеет доступ к странице модерации (рисунок 32) по ссылке «Модерация» в навигационном меню. На странице представлены три раздела:

- «На модерации» – треки, ожидающие решения;
- «Одобрённые треки» – список опубликованных треков;
- «Отклонённые треки» – треки, не прошедшие модерацию.

Для каждого трека администратор может:

- прослушать аудиофайл;
- оставить комментарий;
- одобрить трек (кнопка «Одобрить»);
- отклонить трек (кнопка «Отклонить»).

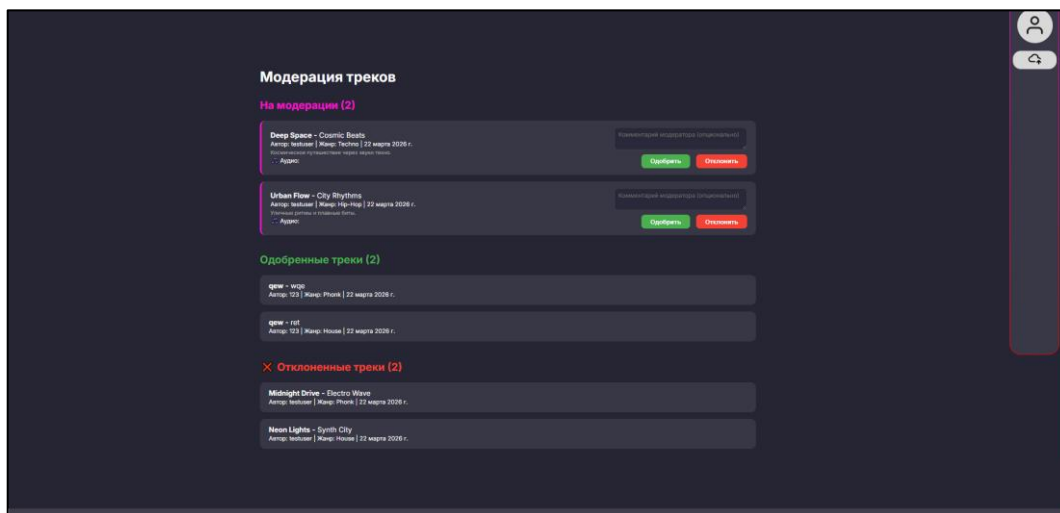


Рисунок 32 – Страница модерации треков

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы был разработан музыкальный блог – веб-приложение, предназначенное для публикации, хранения и оценки музыкальных треков пользователями.

В рамках предпроектного исследования была изучена предметная область музыкальных онлайн платформ, определены основные категории пользователей (гость, зарегистрированный пользователь, администратор) и их функциональные возможности. Проведён анализ инструментальных средств разработки, в результате которого для реализации серверной части был выбран язык программирования Python и веб Фреймворк Django, для хранения данных – система управления базами данных MySQL, для разработки интерфейса – HTML, CSS и JavaScript.

На этапе проектирования были разработаны структурные и функциональные схемы музыкального блога, включая диаграмму прецедентов, диаграмму взаимодействия, диаграмму деятельности, диаграмму классов и диаграмму потоков данных. Спроектирована инфологическая и даталогическая модели базы данных, содержащие шесть таблиц: Roles, Users, Genres, Tracks, Ratings. База данных приведена к третьей нормальной форме, что обеспечивает целостность и минимальную избыточность хранимых данных.

В процессе разработки был реализован следующий функционал:

- регистрация и авторизация пользователей с хешированием паролей;
- разграничение прав доступа между гостем, пользователем и администратором;
- публикация музыкальных треков с загрузкой аудиофайлов;
- система модерации контента, позволяющая администратору одобрять или отклонять треки с возможностью оставления комментария;
- система оценки треков по 10 балльной шкале с автоматическим пересчётом среднего рейтинга;
- поиск треков по названию и имени автора в реальном времени;
- фильтрация треков по жанру;

- личный кабинет пользователя с отображением информации и статуса опубликованных треков;
- страница модерации для администратора с разделением треков по статусам.

Разработанный музыкальный блог соответствует требованиям технического задания, предъявляемым к функциональности, производительности и информационной безопасности. Интерфейс приложения интуитивно понятен, адаптирован для работы в современных браузерах и обеспечивает удобное взаимодействие пользователей с системой.

Таким образом, поставленная цель курсовой работы – разработка музыкального блога для публикации и оценки музыкальных треков – достигнута в полном объёме. Все поставленные задачи успешно решены, программный продукт готов к использованию.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- 1 Django Project – официальная документация Django – URL: <https://docs.djangoproject.com/en/4.2/> (дата обращения 10.03.2026). – Текст: электронный.
- 2 Habr – Разработка веб-приложений на Django – URL: <https://habr.com/ru/companies/tinkoff/articles/790466/> (дата обращения 10.03.2026). – Текст: электронный.
- 3 Habr – Сравнение Django и Flask – URL: <https://habr.com/ru/articles/500876/> (дата обращения 09.03.2026). – Текст: электронный.
- 4 JetBrains – официальная документация PyCharm – URL: <https://www.jetbrains.com/help/pycharm/> (дата обращения 17.03.2026). – Текст: электронный.
- 5 Learn.JavaScript – Основы JavaScript – URL: <https://learn.javascript.ru/> (дата обращения 18.03.2026). – Текст: электронный.
- 6 MDN Web Docs – документация по веб-технологиям – URL: <https://developer.mozilla.org/ru/> (дата обращения 25.03.2026). – Текст: электронный.
- 7 Python.org – официальная документация Python – URL: <https://docs.python.org/3/> (дата обращения 12.03.2026). – Текст: электронный.
- 8 Skillbox – Что такое Django и зачем его изучать – URL: https://skillbox.ru/media/code/что_такое_django_i_zachem_ego_izuchat/ (дата обращения 05.03.2026). – Текст: электронный.
- 9 SQL Academy – учебник по SQL – URL: <https://sql-academy.org/ru/guide> (дата обращения 11.03.2026). – Текст: электронный.
- 10 W3Schools – учебник по HTML, CSS, JavaScript – URL: <https://www.w3schools.com/> (дата обращения 14.03.2026). – Текст: электронный.

Приложение А – Техническое задание
Министерство образования Иркутской области
Государственное бюджетное профессиональное
образовательное учреждение Иркутской области
«Иркутский авиационный техникум»
(ГБПОУИО «ИАТ»)

ТЕХНИЧЕСКОЕ ЗАДАНИЕ
БЛОГ МУЗЫКИ

Руководитель:

(подпись, дата)

(М.А. Кудрявцева)

Студент:

(подпись, дата)

(Г.Д. Щемелев)

Иркутск, 2026

1 Введение

1.1 Общие сведения

Настоящий документ представляет собой техническое задание на разработку автоматизированного музыкального блога (далее - МБ), предназначенной для публикации, хранения и оценки музыкальных треков пользователями через веб-интерфейс.

Разрабатываемая система ориентирована на использование в учебных и творческих целях и предназначена для начинающих и независимых исполнителей, а также слушателей.

Вид программного продукта: Блог.

1.2 Цели и задачи

Целью создания музыкального блога является автоматизация процесса размещения музыкальных треков и получения обратной связи от пользователей в виде оценок.

Основные задачи системы включают:

- Организация публикации музыкальных треков;
- Обеспечение возможности прослушивания треков;
- Реализация системы оценки музыкального контента;
- Хранение информации о пользователях и треках;
- Обеспечение удобного и понятного пользовательского интерфейса.

2 Основания для разработки

2.1 Нормативные документы

Документ основывается на следующих нормативных документах:

- ГОСТ 34.6022020 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

- ГОСТ Р 582752018 «Информационные технологии. Веб-сайты. Требования к пользовательским интерфейсам».
- ГОСТ Р 543032011 «Системы обработки информации. Веб-сайты. Общие требования».
- ГОСТ Р 52633.42008 «Информационная технология. Архитектура систем. Управление качеством системного программного обеспечения»

2.2 Проектные документы

Проектные документы включают:

- Задание на курсовую работу.
- Методические указания по выполнению курсовой работы.
- Пояснительная записка к курсовой работе.

3 Назначение системы

3.1 Общее описание

Музыкальный блог предназначен для использования в сети Интернет и обеспечивает пользователям возможность размещать музыкальные треки, прослушивать работы других авторов и оценивать их.

Музыкальный блог реализуется в виде веб-приложения с разделением прав доступа между пользователями и администратором.

4 Требования к системе

4.1 Функциональные требования

- 1 Функционал роли «Гость»:
 - Прослушивание треков.
 - Просмотр блога.
 - Пользование системой навигации.
 - Возможность регистрации.

2 Функционал роли «Пользователь»:

Все функции роли «Гость», а также:

- Выставление оценки трекам.
- Загрузка своих треков.
- Авторизация.
- Управление своими треками (Редактирование, удаление).

3 Функционал Администратора:

- Авторизация.
- Управление учетными записями пользователи (Удаление, редактирование).
- Управление всеми треками в системе (Удаление).

4.2 Технические требования

4.2.1 Производительность:

- Количество одновременных пользователей: до 15 активных сессий;
- Количество запросов в секунду (RPS/QPS): 200 RPS;
- Среднее время отклика: не более 3 мс.

4.2.2 Надежность:

- Валидация пользовательского ввода на клиенте - все формы, поля, файлы и параметры должны проверяться перед отправкой запроса на сервер.
- Валидация пользовательского ввода на сервере - все входные данные, независимо от наличия клиентской проверки, проходят повторную строгую валидацию.

4.3 Эксплуатационные требования

1 Среда функционирования:

- Поддержка современных браузеров: Яндекс браузер, Chrome, Edge.

2 Резервирование и восстановление:

- Ежедневное резервное копирование базы данных.
- Инструкция восстановления базы данных из резервной копии.

4.4 Требования к информационной безопасности

Система безопасности музыкального блога должна обеспечивать защиту данных и предотвращение несанкционированного доступа в соответствии со следующими требованиями:

1. Аутентификация и авторизация:

– Система должна обеспечивать аутентификацию пользователей перед предоставлением доступа к функциональным модулям.

– Пароли пользователей должны храниться в зашифрованном виде.

– Для каждой пользовательской сессии должен формироваться уникальный токен. Токен должен иметь ограниченное время жизни 30 минут и быть защищён от подделки.

2. Защита от распространённых уязвимостей:

– Защита от несанкционированного доступа к данным - проверка прав доступа к каждому ресурсу на серверной стороне.

5 Требования к техническому обеспечению

5.1 Сервер

- Количество процессоров: 1 шт.
- Количество ядер одного процессора: 4 ядра.
- Тактовая частота одного процессора: 2.4 ГГц.
- Оперативная память: 16 Гб.
- Пропускная способность сети: 500 Мбит/с (Гбит/с).

5.2 Хранилище данных

- Тип накопителей: HDD.
- Общий объем хранилища: 256 Гб.
- IOPS на чтение: 10000.
- IOPS на запись: 5000.

5.3 Клиентские устройства

5.3.1 Минимальные требования для персонального компьютера (ноутбука)

- Количество ядер процессора: 2 ядра.
- Тактовая частота процессора: 1.8 ГГц.
- Оперативная память: 4 Гб.
- Тип диска: HDD или SSD.
- Свободного места на диске: не менее 1 Гб.
- Разрешение дисплея: Full HD (1920×1080).
- Пропускная способность сети: 10 Мбит/с.

5.4 Сетевые требования

- Доступ к сети Интернет со скоростью не менее 10 Мбит/с
- Поддержка следующих сетевых протоколов: HTTP/HTTPS, TCP/IP.
- Поддержка IPv4 (обязательно).
- Возможность работы через защищённые порты 443 (HTTPS) и 80 (HTTP).

6 Требования к программному обеспечению

6.1 Сервер

- Операционная система Windows Server 2022 Standard.
- Сервер базы данных: MySQL 8.4 LTS
- Веб-сервер: Apache HTTP Server 2.4.58.

- Интерпретатор: Python 3.12

6.2 Персональный компьютер (ноутбук)

1. Операционная система Windows 10 / 11 (64bit).

2. Веббраузеры (актуальные версии, не старше двух релизов):

Яндекс.Браузер 24.9+, Google Chrome 128, Microsoft Edge 128+.

7 Организационно-технические требования

7.1 Этапы разработки

В таблице 1 представлены сроки и этапы разработки музыкального блога.

Таблица 1 – Сроки и этапы разработки музыкального

№	Этап	Срок выполнения
1	Предпроектное исследование предметной области	до 17.02.2025
2	Разработка технического задания	до 22.02.2025
3	Проектирование программного обеспечения	до 14.03.2025
4	Разработка (программирование) и отладка программного продукта	до 30.04.2025
5	Составление программной документации	до 14.05.2025

Приложение Б – Листинг представления публикации трека

```
from django.shortcuts import render, redirect
from django.contrib import messages
from .models import Track, Genre
def upload_track(request):
    """Публикация трека - отправляется на модерацию"""
    if 'user_id' not in request.session:
        messages.error(request, 'Сначала войдите в систему')
        return redirect('/login/')
    if request.method == 'POST':
        title = request.POST.get('track_name')
        author_name = request.POST.get('artist_name')
        genre_id = request.POST.get('genre')
        description = request.POST.get('description', '')
        audio_file = request.FILES.get('audio_file')
        if not title or not author_name or not genre_id:
            messages.error(request, 'Заполните все обязательные поля')
            return redirect('/upload/')
        if not audio_file:
            messages.error(request, 'Выберите аудиофайл')
            return redirect('/upload/')
        try:
            track = Track.objects.create(
                title=title,
                author_name=author_name,
                description=description,
                user_id=request.session['user_id'],
                genre_id=genre_id,
                audio_file=audio_file,
                status='pending'
            )
            messages.success(request, 'Трек отправлен на модерацию!')
            return redirect('/profile/')
        except Exception as e:
            messages.error(request, f'Ошибка при публикации: {str(e)}')
    genres = Genre.objects.all()
    context = {'genres': genres}
    return render(request, 'upload.html', context)
```